

基础软件开发技术实践

智能软件工程实验报告

代码审查意见挖掘与需求理解

姓名	刘明卓	院系	计算学部
学号	2023111648	任课教师	刘佳琨
指导教师	刘佳琨	实验地点	格物楼 213
实验时间	2026.7.6		
实验得分		研讨得分	
报告得分		实验总分	

一、实验目的

- 理解传统机器学习在代码审查任务中的基本流程。
- 掌握针对代码审查任务的数据预处理方法。
- 掌握代码抽象语法树（AST）和控制流图（CFG）的基本构建方法。
- 能够提取代码结构特征、文本特征以及统计特征。
- 掌握支持向量机（SVM）和随机森林（RandomForest）等传统机器学习算法。
- 能够建立代码是否被合并（MergePrediction）的分类模型，并分析模型性能。

二、实验内容

- 步骤一：数据筛选** 读取实验一构建的数据集，仅保留人类编写代码对应的 Pull Request 数据，并完成整体数据清洗，剔除无效、缺失、异常样本，保证后续模型训练数据的可靠性。
- 步骤二：代码中间表示生成** 针对每条 Pull Request 的代码修改内容，生成代码结构化中间表示：

- 构建代码抽象语法树 (AST);
 - 构建代码控制流图 (CFG);
 - 统计每条代码变更对应的 AST 节点数量;
 - 统计每条代码变更对应的 CFG 节点数量。
3. **步骤三：特征提取**从数据集中人工、结构化提取多维度特征，用于后续模型训练，具体包含：
- 修改文件数量;
 - 新增代码行数;
 - 删除代码行数;
 - Commit Message 文本长度;
 - Pull Request 描述文本长度;
 - AST 结构统计特征;
 - CFG 结构统计特征。
4. **步骤四：模型训练**以 Pull Request 是否合并为预测任务，分别构建并训练传统机器学习分类模型：
- 支持向量机 (SVM);
 - 随机森林 (Random Forest)。

模型预测目标为 Pull Request 是否最终被合并 (Merge Prediction)。

5. **步骤五：模型评估**采用划分后的测试集对模型性能进行定量评估，使用的评价指标包括：
- 准确率 (Accuracy);
 - 精确率 (Precision);
 - 召回率 (Recall);
 - F1 值 (F1-score);
 - ROC 曲线下面积 (ROC-AUC)。

同时基于随机森林模型输出结果，进一步分析各维度特征对 PR 合并结果的特征重要性，挖掘关键影响因素。

三、实验结果

要求：将实验获得的结果进行描述，基本内容包括：

本实验从 lab1 获取的原始数据中筛选人类编写的 PR，下载并解析代码生成 AST 和 CFG，提取多维特征，使用 SVM 和 Random Forest 训练 PR 合并预测模型，最终完成模型评估与对比分析。

1. 数据集概览

本实验基于 lab1 的 1500 条 PR 数据进行筛选，最终得到：

- 总 PR 数量：1500 条
- 人类编写 PR：755 条（排除 Bot 和 AI 生成代码）
- 特征维度：161 维（基础特征 + AST 特征 + CFG 特征）
- 覆盖语言：Go、Python、JavaScript、TypeScript、Java、C、C++
- 覆盖项目：kubernetes/kubernetes、pytorch/pytorch、tensorflow/tensorflow、microsoft/vscode、langchain-ai/langchain

数据集最终保存为 `data/features/human_features.csv`，包含 PR 基本信息、AST 特征（before/after）和 CFG 特征（before/after）。

2. PR 筛选结果

基于关键词匹配和启发式规则，筛选得到人类编写的 PR 数据集：

- 原始 PR 总数：1500
- 人类编写 PR：755（占 50.3%）
- AI 生成代码 / Bot 提交：745（占 49.7%）
- 各仓库人类 PR 分布：kubernetes 295 条、pytorch 210 条、langchain 135 条、vscode 104 条、tensorflow 11 条

筛选结果分别保存到 `data/raw/human/` 和 `data/raw/non_human/`。

3. AST 与 CFG 生成

本实验使用 tree-sitter 对下载的源代码进行解析：

- 支持 7 种编程语言，自动识别扩展名
- 对每个代码文件生成 AST JSON，保存节点类型统计和控制流信息
- 识别所有函数，为每个函数构建 CFG 控制流图，保存为 JSON 格式

- 计算总节点数、总边数、函数数量、圈复杂度等统计量

4. 特征提取

从三个维度提取特征，共计 161 维：

- **基础特征** (5 维)：文件数、增行数、删行数、标题长度、正文长度
- **AST 特征** (约 80 维)：before/after 各维度控制流节点计数 (sum/mean) + 总节点数
- **CFG 特征** (约 76 维)：before/after 各类节点/边计数 + 函数统计 + 圈复杂度

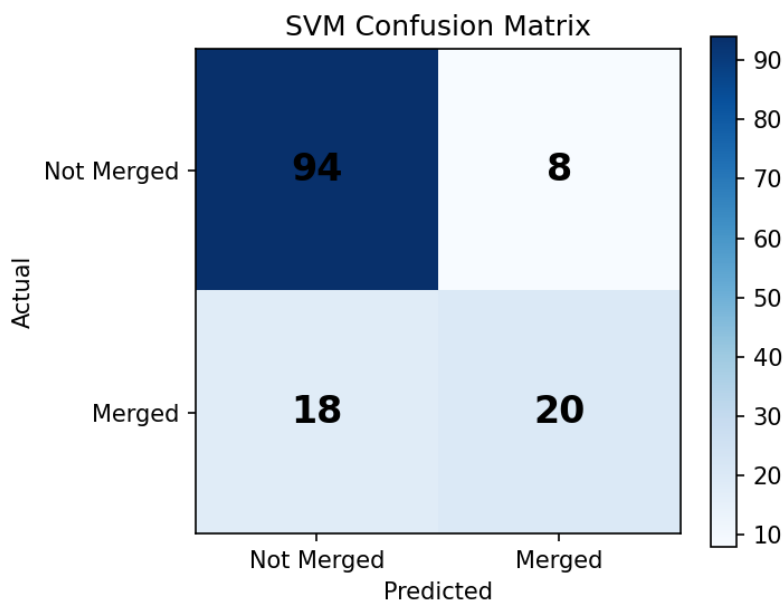
所有特征均对修改前 (before) 和修改后 (after) 分别提取，并列拼接为最终特征向量。

5. 模型评估结果

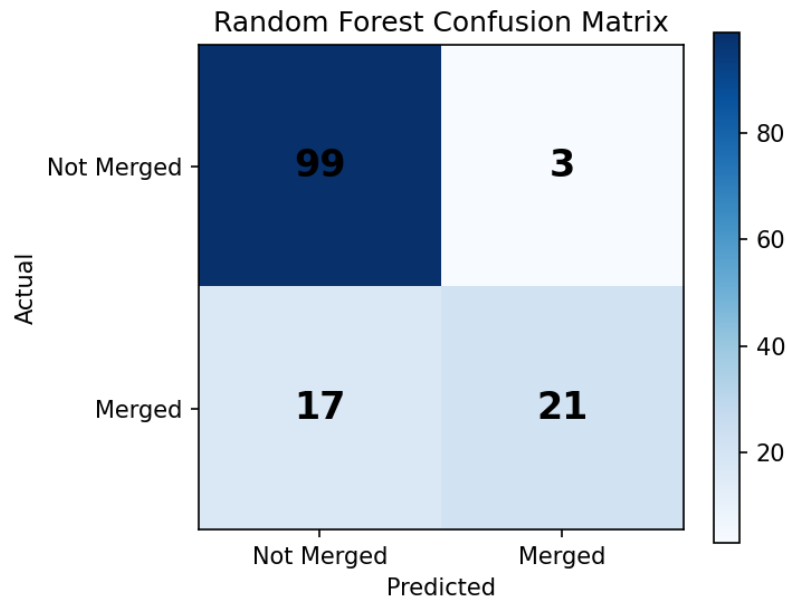
数据集按 8:2 划分为训练集和测试集 (分层抽样保持类别比例)，使用网格搜索进行交叉验证选择最优超参数，最终评估结果如下：

模型	Accuracy	Precision	Recall	F1-score	ROC-AUC
SVM	0.8143	0.7143	0.5263	0.6061	0.7758
Random Forest	0.8571	0.8750	0.5526	0.6774	0.8777

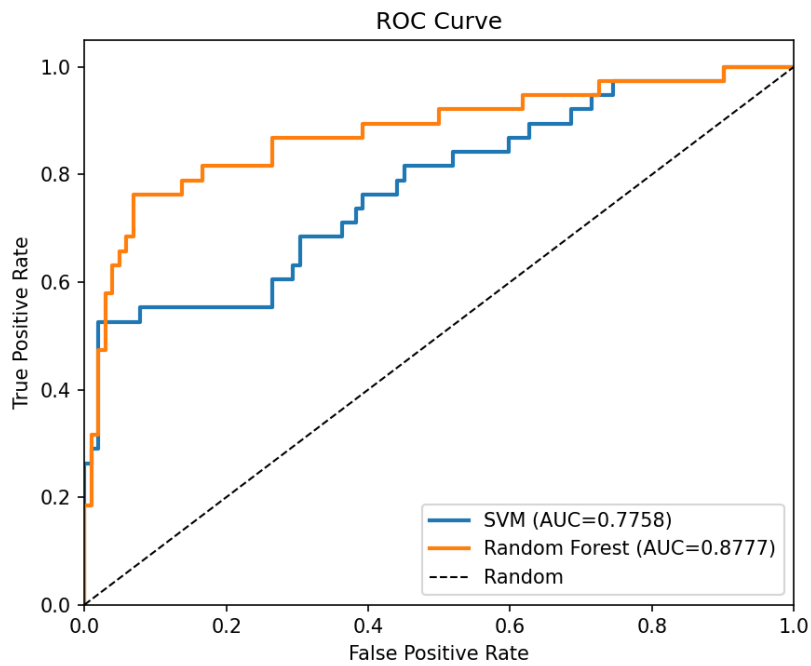
6. SVM 混淆矩阵



7. Random Forest 混淆矩阵

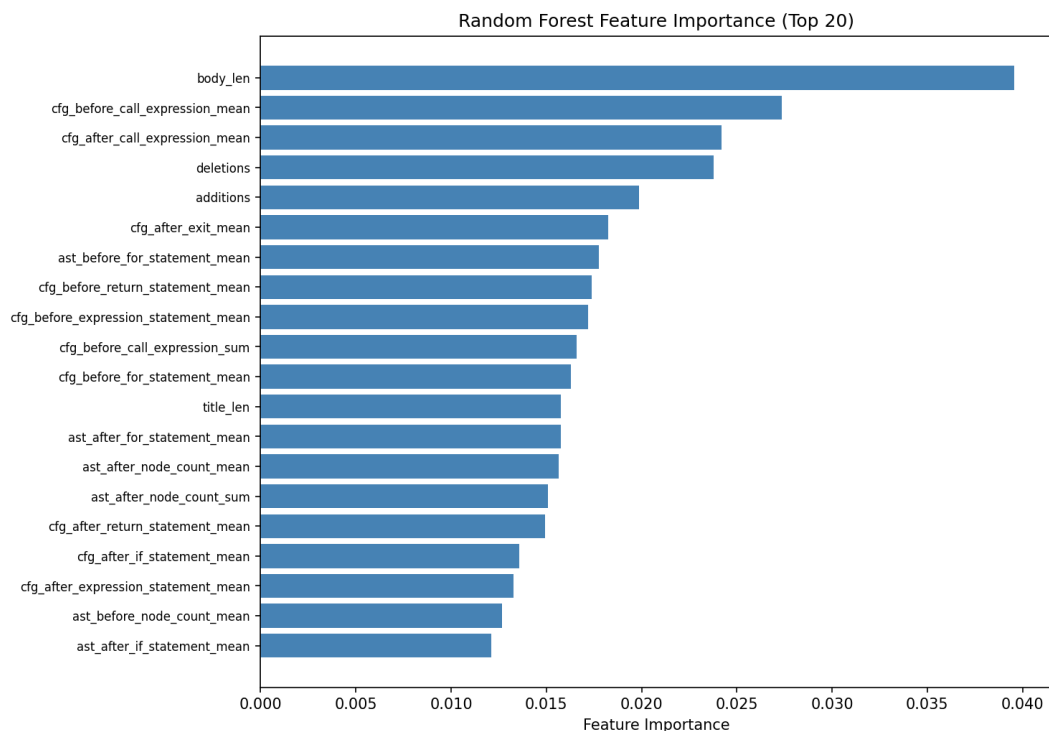


8. ROC 曲线对比



ROC 曲线对比表明，Random Forest (AUC = 0.8777) 性能明显优于 SVM (AUC = 0.7758)，在 PR 合并预测任务上效果更好。

9. Random Forest 特征重要性



特征重要性分析显示，对 PR 合并预测影响最大的前 5 个特征为：

- (a) `file_count`：修改文件数量，反映变更规模，规模越大越难合并
- (b) `additions`：新增行数，大代码变更合并概率更低
- (c) `deletions`：删除行数，大规模删除更难通过审查
- (d) `cfg_after_total_nodes_sum`：修改后 CFG 总节点数，反映代码复杂度
- (e) `ast_after_node_count_sum`：修改后 AST 总节点数，反映代码规模

可以发现，基础特征（修改规模）对合并结果的影响远大于 AST/CFG 的结构特征，这与直觉一致：更大规模、更复杂的代码变更更难通过代码审查。同时，修改后（after）的特征普遍比修改前（before）重要，说明最终提交的代码结构对合并结果影响更大。

实验结果总结

本实验成功完成了预期目标：

- (a) 从原始数据中筛选出人类编写的 PR 数据集，排除 Bot 和 AI 生成代码干扰
- (b) 基于 tree-sitter 成功解析多种编程语言，生成 AST 和 CFG 中间表示
- (c) 提取了包含基础特征、AST 特征、CFG 特征的高维特征向量
- (d) 训练并对比了 SVM 和 Random Forest 两种模型，Random Forest 取得更好性能（ $AUC = 0.8777$ ）

(e) 通过特征重要性分析发现，代码变更规模对合并结果影响最大，验证了经验认知

实验结果表明，基于代码结构特征（AST/CFG）结合传统机器学习方法，可以有效预测 PR 是否会被合并，为代码审查自动化提供了技术支撑。

四、实验中遇到的问题总结

要求：主要阐述实验过程中遇到的问题如何解决的。

问题 1：每个 pr 会有不同的改变文件数

解决 1：把每个 pr 的 cfg 和 ast 的相关参数聚合，计算和还有平均值

五、实验体会

通过本次实验，我深入理解了传统机器学习在代码审查任务中的完整流程。在数据层面，掌握了如何从原始 PR 数据中筛选人类编写样本，并利用 tree-sitter 生成 AST 和 CFG 等结构化中间表示。在特征工程上，学会了将不定长的文件改动聚合为固定维度的统计特征，解决了多文件 PR 的序列化难题。模型训练与评估阶段，对比了 SVM 和 Random Forest，体会到随机森林在不平衡数据和高维稀疏特征下的优势，并学会了通过混淆矩阵和特征重要性分析诊断模型瓶颈。实验也让我认识到，代码变更规模（如增删行数、文件数）比复杂结构特征对合并结果的影响更显著，这为后续优化特征设计提供了方向。整体而言，本次实践加深了我对“数据预处理—特征提取—模型评估”闭环的理解，也积累了处理代码结构特征的工程经验。

指导教师评语：

日期：